JAVA - Collections

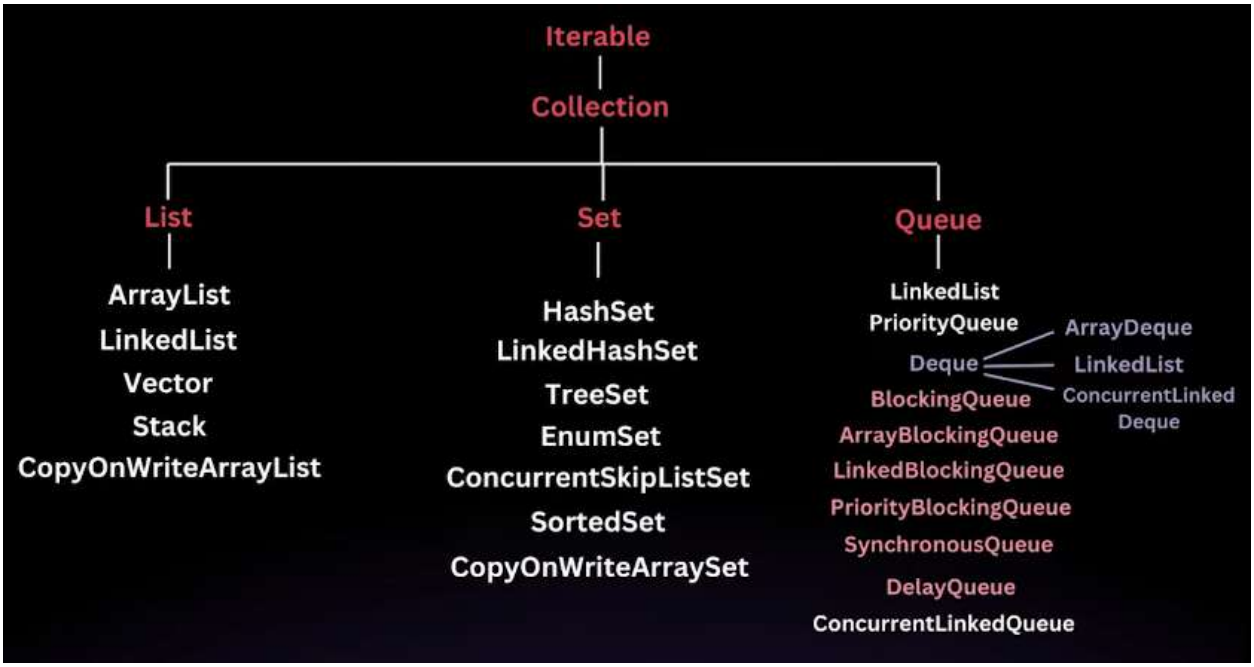
What is collection?

Collection is Simply an Object that represents a group of objects, known as its elements.

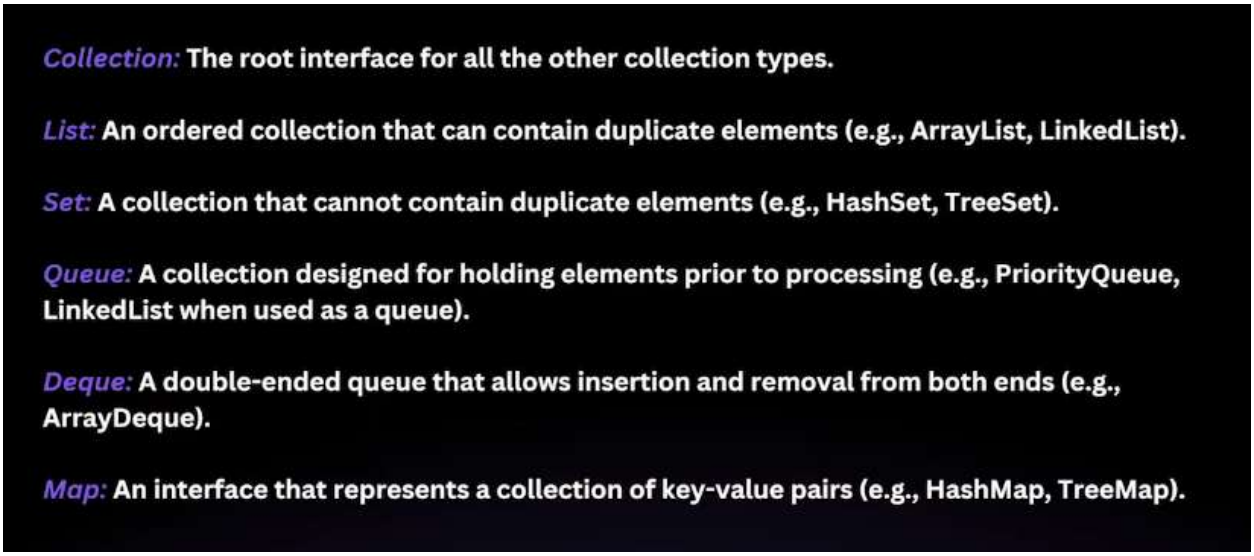
What is Collection Framework?

It Provides a set of interfaces and classes that help in managing groups of objects

Collection Hierarchy



Collections



Collection Hierarchy

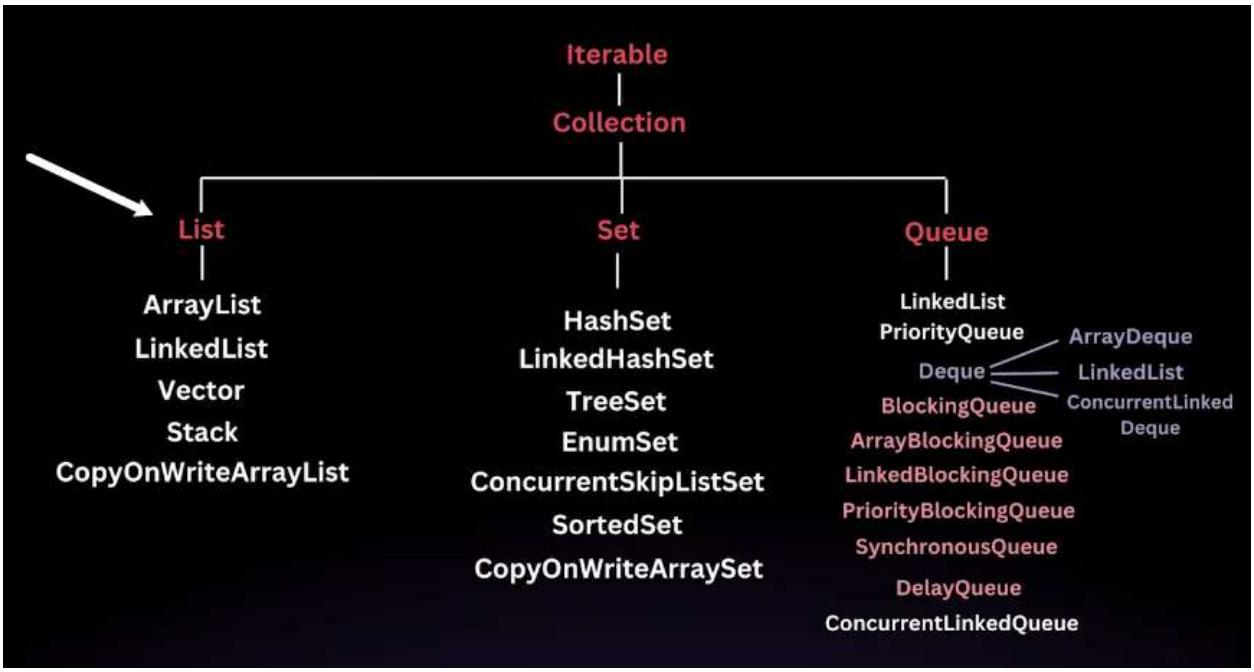
The Collection Framework is organized into a hierarchy where the core interfaces are at the top, and the specific implementations extend these interfaces.

The Collection interface is the *root interface* of the Java Collection Framework. It is the most basic interface that defines a group of objects known as elements. The Collection interface is a part of the java.util package, and *It is a parent interface that is extended by other collection interfaces like List, Set, and Queue*.

Since Collection is an interface, it cannot be instantiated directly; rather, it provides a blueprint for the basic operations that are common to all collections.

The Collection interface defines a set of core methods that are implemented by all classes that implement the interface. These methods allow for basic operations such as adding, removing, and checking the existence of elements in a collection.

List Interface



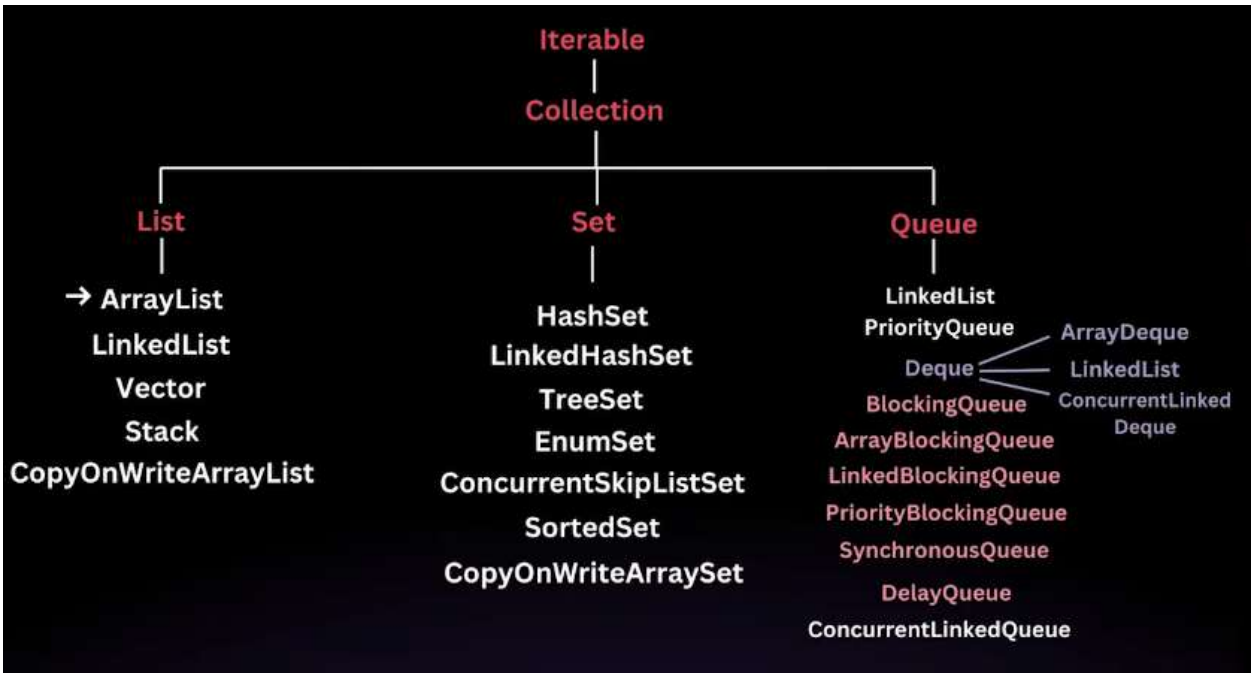
The List interface in Java is a part of the java.util package and is a sub-interface of the Collection interface. It provides a way to store an *ordered collection of elements* (known as a sequence). Lists allow for precise control over where elements are inserted and *can contain duplicate elements.*

The List interface is implemented by several classes in the Java Collection Framework, such as *ArrayList, LinkedList, Vector, and Stack*.

Key Features of the List Interface

Order Preservation
Index-Based Access
Allows Duplicates

Array List



An ArrayList is a resizable array implementation of the List interface. Unlike arrays in Java, which have a fixed size, an *ArrayList can change its size dynamically* as elements are added or removed. This flexibility makes it a popular choice when the number of elements in a list isn't known in advance.

Internal working

Unlike a regular array, which has a fixed size, an ArrayList can grow and shrink as elements are added or removed. This dynamic resizing is achieved by creating a new array when the current array is full and copying the elements to the new array.

When you create an ArrayList, it has an initial capacity (default is 10). The capacity refers to the size of the internal array that can hold elements before needing to resize.

Adding Elements

When we add an element to an ArrayList, the following steps occur

Check Capacity: Before adding the new element, ArrayList checks if there is enough space in the internal array (elementData). If the array is full, it needs to be resized.

Resize if Necessary: If the internal array is full, the ArrayList will create a new array with a larger capacity (usually 1.5 times the current capacity) and copy the elements from the old array to the new array.

Add the Element: The new element is then added to the internal array at the appropriate index, and the size is incremented.

Resizing the Array

- *Initial Capacity:* By default, the initial capacity is 10. This means the internal array can hold 10 elements before it needs to grow.
- *Growth Factor:* When the internal array is full, a new array is created with a size 1.5 times the old array. This growth factor balances memory efficiency and resizing cost.
- *Copying Elements:* When resizing occurs, all elements from the old array are copied to the new array, which is an O(n) operation, where n is the number of elements in the ArrayList.

Creating an ArrayList

```java
// Default constructor, creates an empty ArrayList with an initial capacity of 10
ArrayList<String> list = new ArrayList<>();

// Creating an ArrayList with a specified initial capacity
ArrayList<String> listWithCapacity = new ArrayList<>(20);

// Creating an ArrayList from another collection
List<String> anotherList = Arrays.asList("Apple", "Banana", "Orange");
ArrayList<String> listFromCollection = new ArrayList<>(anotherList);
```

```java
// ==============================
// 1. Creating ArrayList
// ==============================

ArrayList<Integer> list = new ArrayList<>();
ArrayList<Integer> list2 = new ArrayList<>(10); // initial capacity
ArrayList<Integer> list3 = new ArrayList<>(Arrays.asList(1,2,3,4,5));

System.out.println("Initial List: " + list);
```

```java
// ==============================
// 2. Adding Elements
// ==============================

list.add(10);
list.add(20);
list.add(30);

list.add(1, 15); // add at index

list.addAll(Arrays.asList(40,50,60));

System.out.println("After Add: " + list);
```

```java
// ==============================
// 3. Accessing Elements
// ==============================

int first = list.get(0);
int second = list.get(1);

System.out.println("First Element: " + first);

// ==============================
// 4. Updating Elements
// ==============================

list.set(0, 100);

System.out.println("After Update: " + list);
```

```java
// ==============================
// 5. Removing Elements
// ==============================

list.remove(2); // remove by index

list.remove(Integer.valueOf(50)); // remove by value

System.out.println("After Remove: " + list);
```

```java
// ==============================
// 7. Checking Elements
// ==============================

boolean contains = list.contains(30);

System.out.println("Contains 30? " + contains);
```


```java
// ==============================
// 7. Checking Elements
// ==============================

boolean contains = list.contains(30);

System.out.println("Contains 30? " + contains);
```

```java
// ==============================
// 8. Checking Empty
// ==============================

boolean empty = list.isEmpty();

System.out.println("Is Empty: " + empty);

// ==============================
// 9. Searching Elements
// ==============================

int index = list.indexOf(40);
int lastIndex = list.lastIndexOf(40);

System.out.println("Index of 40: " + index);
```

```java
// ==============================
// 10. Iterating ArrayList
// ==============================

System.out.println("For Loop:");
for(int i=0;i<list.size();i++)
    System.out.println(list.get(i));

System.out.println("For Each Loop:");
for(Integer num : list)
    System.out.println(num);

System.out.println("Iterator:");
Iterator<Integer> iterator = list.iterator();

while(iterator.hasNext())
    System.out.println(iterator.next());

System.out.println("ListIterator:");

ListIterator<Integer> listIterator = list.listIterator();

while(listIterator.hasNext())
    System.out.println(listIterator.next());
```

```
    // ==============================
    // 11. Sorting
    // ==============================

    Collections.sort(list);

    System.out.println("Sorted Asc: " + list);

    Collections.sort(list, Collections.reverseOrder());

    System.out.println("Sorted Desc: " + list);

    // ==============================
    // 12. Reversing
    // ==============================

    Collections.reverse(list);

    System.out.println("Reversed: " + list);

    // ==============================
    // 13. Shuffling
    // ==============================

    Collections.shuffle(list);

    System.out.println("Shuffled: " + list);
```

```
// ==============================
// 14. SubList
// ==============================

List<Integer> subList = list.subList(1,3);

System.out.println("SubList: " + subList);
```

```java
// ==============================
// 15. Converting ArrayList
// ==============================

Object[] array = list.toArray();

Integer[] intArray = list.toArray(new Integer[0]);

// ==============================
// 16. Cloning
// ==============================

ArrayList<Integer> clonedList = (ArrayList<Integer>) list.clone();

System.out.println("Cloned List: " + clonedList);

// ==============================
// 17. Clearing List
// ==============================

list.clear();

System.out.println("After Clear: " + list);
```

```java
// ==============================
// 18. Capacity Operations
// ==============================

ArrayList<Integer> capacityList = new ArrayList<>(20);

capacityList.addAll(Arrays.asList(1,2,3,4));

capacityList.trimToSize(); // reduce capacity to size

capacityList.ensureCapacity(50); // increase capacity

System.out.println("Capacity Operations Done");
```

```java
// ==============================
// 19. Replace All
// ==============================

list3.replaceAll(n -> n*2);

System.out.println("ReplaceAll: " + list3);

// ==============================
// 20. Remove If
// ==============================

list3.removeIf(n -> n > 6);

System.out.println("RemoveIf: " + list3);

// ==============================
// 21. Retain All
// ==============================

ArrayList<Integer> retainList = new ArrayList<>(Arrays.asList(2,4,6));

list3.retainAll(retainList);

System.out.println("RetainAll: " + list3);
```

```java
// ==============================
// 22. Contains All
// ==============================

boolean containsAll = list3.containsAll(retainList);

System.out.println("ContainsAll: " + containsAll);
```

Time Complexity

- Access by index (get) is O(1).
- Adding an element is O(n) in the worst case when resizing occurs.
- Removing elements can be O(n) because it may involve shifting elements.
- Iteration is O(n).

Comparator– Complete Flow

Why Comparator?

Comparator is used to **define custom sorting logic outside a class**.

It is commonly used with:

- Collections.sort()
- List.sort()
- Stream.sorted()
- TreeSet
- TreeMap

PART 1 — Comparator with Primitive Collections

Sorting Integer List

```
List<Integer> nums = new ArrayList<>(Arrays.asList(5,2,8,1,9,3));

Collections.sort(nums);
```

Sorting Integer Descending

```
Collections.sort(nums, Comparator.reverseOrder());
```

Custom Integer Comparator

```
Collections.sort(nums,(a,b)->b-a);
```

Descending order using lambda.

Sort Even Numbers First

```
Collections.sort(nums,(a,b)->{
    if(a%2==0 && b%2!=0) return -1;
    if(a%2!=0 && b%2==0) return 1;
    return a-b;
});
```

PART 2 — Comparator with String Collections

Alphabetical Sorting

```
List<String> names = new ArrayList<>(Arrays.asList("Surya","Mahesh","Kiran","Anil"));

Collections.sort(names);
```

Reverse Alphabetical Sorting

```
Collections.sort(names, Comparator.reverseOrder());
```

Sort by String Length

```
Collections.sort(names, Comparator.comparing(String::length));
```

Sort by String Length Desc

```
Collections.sort(names,
        Comparator.comparing(String::length).reversed());
```

Custom Comparator (Lambda)

```
Collections.sort(names,(a,b)->a.length()-b.length());
```

PART 3 — Comparator with Objects

```
class Employee {

    int id;
    String name;
    int age;
    double salary;

    public Employee(int id,String name,int age,double salary){
        this.id=id;
        this.name=name;
        this.age=age;
        this.salary=salary;
    }

    public int getId(){return id;}
    public String getName(){return name;}
    public int getAge(){return age;}
    public double getSalary(){return salary;}

    public String toString(){
        return id+" "+name+" "+age+" "+salary;
    }
}
```

```
List<Employee> employees = new ArrayList<>();

employees.add(new Employee(1,"Surya",25,60000));
employees.add(new Employee(2,"Mahesh",30,50000));
employees.add(new Employee(3,"Kiran",28,75000));
employees.add(new Employee(4,"Anil",35,90000));
```

Common Employee Sorting Examples

Sort by Age

```
Collections.sort(
        employees,
        Comparator.comparing(Employee::getAge)
);
```

Sort by Salary

```
Collections.sort(
        employees,
        Comparator.comparing(Employee::getSalary)
);
```

Sort by Salary Descending

```
Collections.sort(
        employees,
        Comparator.comparing(Employee::getSalary).reversed()
);
```

Sort by Name

```
Collections.sort(
        employees,
        Comparator.comparing(Employee::getName)
);
```

Multi Field Sorting

Sort by **Age then Salary**

```
Collections.sort(
        employees,
        Comparator.comparing(Employee::getAge)
        .thenComparing(Employee::getSalary)
);
```

Multi Field Sorting (Advanced)

Salary Desc → Age Asc

```
Collections.sort(
        employees,
        Comparator.comparing(Employee::getSalary)
        .reversed()
        .thenComparing(Employee::getAge)
);
```

Custom Comparator Class

```
class SalaryComparator implements Comparator<Employee>{

    public int compare(Employee e1,Employee e2){
        return Double.compare(e1.getSalary(),e2.getSalary());
    }
}
```

Usage:

```
Collections.sort(employees,new SalaryComparator());
```

Sort Integers by Absolute Value

```
class AbsComparator implements Comparator<Integer>
{
    public int compare(Integer a,Integer b)
    {
        return Math.abs(a) - Math.abs(b);
    }
}

nums.sort(new AbsComparator());
```

Sort Integers Even First Then Odd

```
class EvenFirstComparator implements Comparator<Integer>
{
    public int compare(Integer a,Integer b)
    {
        if(a%2==0 && b%2!=0) return -1;
        if(a%2!=0 && b%2==0) return 1;
        return a-b;
    }
}

nums.sort(new EvenFirstComparator());
```

Sort Integers by Last Digit

```
class LastDigitComparator implements Comparator<Integer>
{
    public int compare(Integer a,Integer b)
    {
        return (a%10) - (b%10);
    }
}

nums.sort(new LastDigitComparator());
```

Sort Strings Alphabetically (Custom)

```
class AlphabetComparator implements Comparator<String>
{
    public int compare(String s1,String s2)
    {
        return s1.compareTo(s2);
    }
}

names.sort(new AlphabetComparator());
```

Sort Strings by Length Descending

```
class LengthDescComparator implements Comparator<String>
{
    public int compare(String s1,String s2)
    {
        return s2.length() - s1.length();
    }
}

names.sort(new LengthDescComparator());
```

Sort Strings by Last Character

```
class LastCharComparator implements Comparator<String>
{
    public int compare(String s1,String s2)
    {
        return s1.charAt(s1.length()-1) - s2.charAt(s2.length()-1);
    }
}

names.sort(new LastCharComparator());
```

Sort Employees by Salary

```
class SalaryComparator implements Comparator<Employee>
{
    public int compare(Employee e1,Employee e2)
    {
        return Double.compare(e1.getSalary(),e2.getSalary());
    }
}

employees.sort(new SalaryComparator());
```

Sort Employees by Age Descending

```
class AgeDescComparator implements Comparator<Employee>
{
    public int compare(Employee e1,Employee e2)
    {
        return e2.getAge() - e1.getAge();
    }
}

employees.sort(new AgeDescComparator());
```

Sort Employees by Name Length

```
class NameLengthComparator implements Comparator<Employee>
{
    public int compare(Employee e1,Employee e2)
    {
        return e1.getName().length() - e2.getName().length();
    }
}

employees.sort(new NameLengthComparator());
```

Sort Employees by Salary then Age

```
class SalaryAgeComparator implements Comparator<Employee>
{
    public int compare(Employee e1,Employee e2)
    {
        int result = Double.compare(e1.getSalary(),e2.getSalary());

        if(result==0)
            return e1.getAge() - e2.getAge();

        return result;
    }
}

employees.sort(new SalaryAgeComparator());
```

Equivalent lambda version:

```
nums.sort((a,b) -> Math.abs(a) - Math.abs(b));

names.sort((a,b) -> a.length() - b.length());

employees.sort((e1,e2) -> Double.compare(e1.getSalary(),e2.getSalary()));
```

PART 4 — Comparator with Streams

Streams use the **same Comparator logic**.

Sort Employees by Salary

```
List<Employee> sorted =
employees.stream()
.sorted(Comparator.comparing(Employee::getSalary))
.toList();
```

Sort Salary Desc

```
List<Employee> sorted =
employees.stream()
.sorted(
Comparator.comparing(Employee::getSalary).reversed()
)
.toList();
```

Highest Salary Employee

```
Employee highest =
employees.stream()
.max(Comparator.comparing(Employee::getSalary))
.orElse(null);
```

Youngest Employee

```
Employee youngest =
employees.stream()
.min(Comparator.comparing(Employee::getAge))
.orElse(null);
```

Important Comparator Methods

Method	Purpose
`Comparator.comparing()`	compare object fields
`comparingInt()`	optimized primitive comparison
`comparingDouble()`	primitive double comparator
`thenComparing()`	multi-level sorting
`reversed()`	reverse order
`reverseOrder()`	built-in reverse
`nullsFirst()`	null safe sorting
`nullsLast()`	null safe sorting

Comparator vs Comparable

Comparable	Comparator
Inside class	Outside class
Natural order	Custom sorting
Single sorting logic	Multiple sorting strategies

LinkedList

The LinkedList class in Java is a part of the Collection framework and implements the List interface. Unlike an ArrayList, which uses a dynamic array to store the elements, *a LinkedList stores its elements as nodes in a doubly linked list*. This provides different performance characteristics and usage scenarios compared to ArrayList.

Performance Considerations

LinkedList has different performance characteristics compared to ArrayList:

Insertions and Deletions: LinkedList is better for frequent insertions and deletions in the middle of the list because it does not require shifting elements, as in ArrayList.

+

Random Access: LinkedList has slower random access (get(int index)) compared to ArrayList because it has to traverse the list from the beginning to reach the desired index.

Memory Overhead: LinkedList requires more memory than ArrayList because each node in a linked list requires extra memory to store references to the next and previous nodes.

```
// ============================
// 1. Creating LinkedList
// ============================

LinkedList<Integer> list = new LinkedList<>();
LinkedList<Integer> list2 = new LinkedList<>(Arrays.asList(1,2,3,4,5));

System.out.println("Initial List: " + list);
```

```java
// ==============================
// 2. Adding Elements
// ==============================

list.add(10);
list.add(20);
list.add(30);

list.addFirst(5);
list.addLast(40);

list.add(2,15);

list.addAll(Arrays.asList(50,60,70));

System.out.println("After Add: " + list);
```

```java
// ==============================
// 3. Accessing Elements
// ==============================

int first = list.getFirst();
int last = list.getLast();
int element = list.get(2);

System.out.println("First Element: " + first);
System.out.println("Last Element: " + last);


// ==============================
// 4. Updating Elements
// ==============================

list.set(1,100);

System.out.println("After Update: " + list);
```

```java
// ==============================
// 5. Removing Elements
// ==============================

list.remove();
list.removeFirst();
list.removeLast();

list.remove(2);
list.remove(Integer.valueOf(50));

System.out.println("After Remove: " + list);


// ==============================
// 6. Size
// ==============================

int size = list.size();

System.out.println("Size: " + size);
```

```java
 // ==============================
 // 7. Searching Elements
 // ==============================

 boolean contains = list.contains(30);

 int index = list.indexOf(30);

 int lastIndex = list.lastIndexOf(30);

 System.out.println("Contains 30: " + contains);
```

```java
// ==============================
// 8. Iterating LinkedList
// ==============================

System.out.println("For Loop");

for(int i=0;i<list.size();i++)
    System.out.println(list.get(i));
```

```
System.out.println("ForEach Loop");

for(Integer num : list)
    System.out.println(num);


System.out.println("Iterator");

Iterator<Integer> iterator = list.iterator();

while(iterator.hasNext())
    System.out.println(iterator.next());


System.out.println("Descending Iterator");

Iterator<Integer> desc = list.descendingIterator();

while(desc.hasNext())
    System.out.println(desc.next());
```

```
// ==============================
// 9. Queue Operations
// ==============================

LinkedList<Integer> queue = new LinkedList<>();

queue.offer(10);
queue.offer(20);
queue.offer(30);

int head = queue.peek();
int removed = queue.poll();

System.out.println("Queue: " + queue);
```

```java
// ==============================
// 10. Deque Operations
// ==============================

LinkedList<Integer> deque = new LinkedList<>();

deque.offerFirst(10);
deque.offerLast(20);

deque.peekFirst();
deque.peekLast();

deque.pollFirst();
deque.pollLast();

System.out.println("Deque: " + deque);
```

```java
// ==============================
// 11. Stack Operations
// ==============================

LinkedList<Integer> stack = new LinkedList<>();

stack.push(10);
stack.push(20);
stack.push(30);

int top = stack.peek();
stack.pop();

System.out.println("Stack: " + stack);
```

```java
// ==============================
// 12. Sorting
// ==============================

list.addAll(Arrays.asList(3,9,1,7));

Collections.sort(list);

System.out.println("Sorted Asc: " + list);

Collections.sort(list,Collections.reverseOrder());

System.out.println("Sorted Desc: " + list);


// ==============================
// 13. Reversing
// ==============================

Collections.reverse(list);

System.out.println("Reversed: " + list);
```

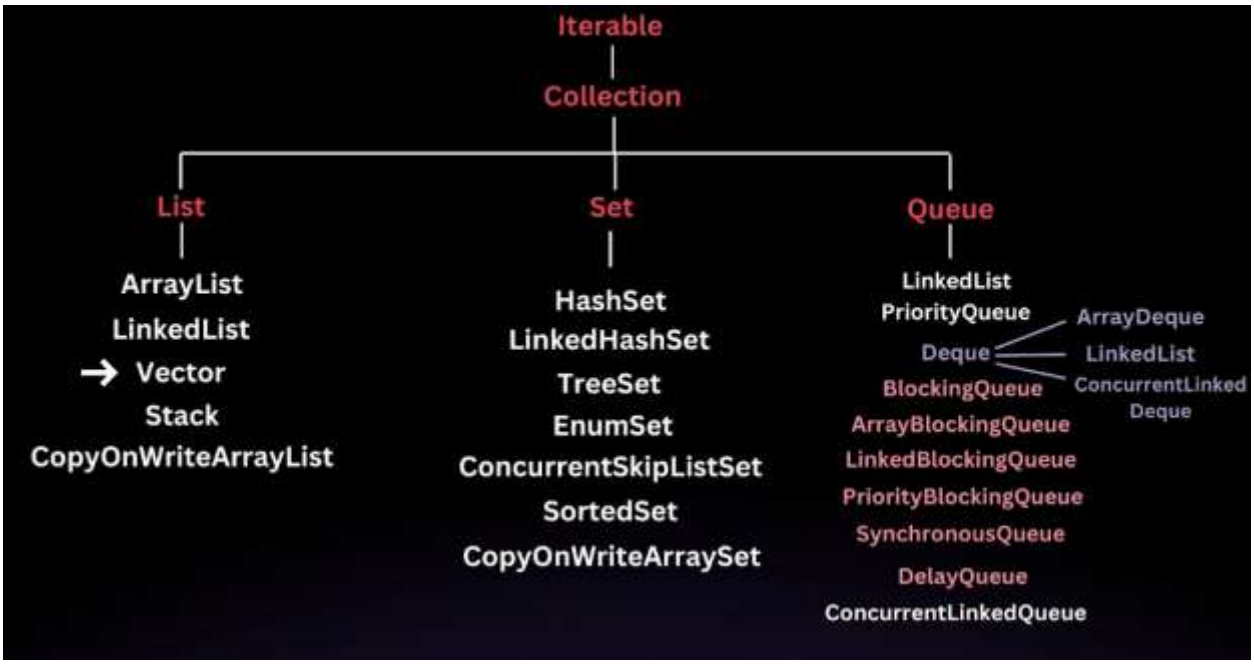
```java
  // ==============================
  // 14. Shuffle
  // ==============================

  Collections.shuffle(list);

  System.out.println("Shuffled: " + list);


  // ==============================
  // 15. SubList
  // ==============================

  List<Integer> subList = list.subList(1,4);

  System.out.println("SubList: " + subList);
```

```java
// ==============================
// 16. Conversion
// ==============================

Object[] arr = list.toArray();

Integer[] arr2 = list.toArray(new Integer[0]);


// ==============================
// 17. Clone
// ==============================

LinkedList<Integer> clonedList = (LinkedList<Integer>) list.clone();

System.out.println("Cloned: " + clonedList);


// ==============================
// 18. Clear List
// ==============================

list.clear();

System.out.println("After Clear: " + list);
```

```java
    // ==============================
    // 19. Remove If
    // ==============================

    list2.removeIf(n -> n % 2 == 0);

    System.out.println("RemoveIf: " + list2);


    // ==============================
    // 20. Replace All
    // ==============================

    list2.replaceAll(n -> n*2);

    System.out.println("ReplaceAll: " + list2);
```

```java
// ==============================
// 21. Retain All
// ==============================

LinkedList<Integer> retain = new LinkedList<>(Arrays.asList(4,8));

list2.retainAll(retain);

System.out.println("RetainAll: " + list2);


// ==============================
// 22. Contains All
// ==============================

boolean containsAll = list2.containsAll(retain);

System.out.println("ContainsAll: " + containsAll);
```

Vector



A Vector in Java is a part of the java.util package and is one of the legacy classes in Java that implements the List interface.

It was introduced in JDK 1.0 before collection framework and is synchronized, making it thread-safe.

Now it is a part of collection framework.

However, due to its synchronization overhead, it's generally recommended to use other modern alternatives like ArrayList in single-threaded scenarios. Despite this, Vector is still useful in certain situations, particularly in multi-threaded environments where thread safety is a concern.

Key Features of Vector

Dynamic Array: Like ArrayList, Vector is a dynamic array that grows automatically when more elements are added than its current capacity.

Synchronized: All the methods in Vector are synchronized, which makes it thread-safe. This means multiple threads can work on a Vector without the risk of corrupting the data. However, this can introduce performance overhead in single-threaded environments.

Legacy Class: Vector was part of Java's original release and is considered a legacy class. It's generally recommended to use ArrayList in single-threaded environments due to performance considerations.

Resizing Mechanism: When the current capacity of the vector is exceeded, it doubles its size by default (or increases by a specific capacity increment if provided).

Random Access: Similar to arrays and ArrayList, Vector allows random access to elements, making it efficient for accessing elements using an index.

Constructors of Vector

Vector(): Creates a vector with an initial capacity of 10.

Vector(int initialCapacity): Creates a vector with a specified initial capacity.

Vector(int initialCapacity, int capacityIncrement): Creates a vector with an initial capacity and capacity increment (how much the vector should grow when its capacity is exceeded).

Vector(Collection<? extends E> c): Creates a vector containing the elements of the specified collection.

Methods in Vector

- *add(E e):* Adds an element at the end.
- *add(int index, E element):* Inserts an element at the specified index.
- *get(int index):* Retrieves the element at the specified index.
- *set(int index, E element):* Replaces the element at the specified index.
- *remove(Object o):* Removes the first occurrence of the specified element.
- *remove(int index):* Removes the element at the specified index.
- *size():* Returns the number of elements in the vector.
- *isEmpty():* Checks if the vector is empty.
- *contains(Object o):* Checks if the vector contains the specified element.
- *clear():* Removes all elements from the vector.

Internal Implementation of Vector

Internally, Vector uses an array to store its elements. The size of this array grows as needed when more elements are added. The default behavior is to double the size of the array when it runs out of space. This resizing operation is a costly one, as it requires copying the old elements to the new, larger array.

Synchronization and Performance

Since Vector methods are synchronized, it ensures that only one thread can access the vector at a time. This makes it thread-safe but can introduce performance overhead in single-threaded environments because synchronization adds locking and unlocking costs.

In modern Java applications, ArrayList is generally preferred over Vector when synchronization isn't required. For thread-safe collections, the CopyOnWriteArrayList or ConcurrentHashMap from the java.util.concurrent package is often recommended instead.

1. Vector is a legacy synchronized collection class that implements the List interface.
2. It behaves like a dynamic array and grows as needed.
3. It provides thread safety but with a performance cost in single-threaded environments.
4. In modern applications, ArrayList or concurrent alternatives like CopyOnWriteArrayList are typically preferred over Vector unless thread safety is a priority.

```
// ================================
// 1. Creating Vector
// ================================

Vector<Integer> vec = new Vector<>();

Vector<Integer> vec2 = new Vector<>(10);

Vector<Integer> vec3 = new Vector<>(Arrays.asList(1,2,3,4,5));

Vector<Integer> vec4 = new Vector<>(10,5); // capacity + increment
```

```
// ================================
// 2. Adding Elements
// ================================

vec.add(10);
vec.add(20);
vec.add(30);

vec.add(1,15);

vec.addElement(40); // legacy method

vec.addAll(Arrays.asList(50,60,70));
```

```
// ================================
// 3. Accessing Elements
// ================================

int first = vec.get(0);

int element = vec.elementAt(2);

int firstElement = vec.firstElement();

int lastElement = vec.lastElement();
```

```
// ================================
// 4. Updating Elements
// ================================

vec.set(0,100);

vec.setElementAt(200,1);
```

```
// ================================
// 5. Removing Elements
// ================================

vec.remove(2);

vec.remove(Integer.valueOf(50));

vec.removeElement(60);

vec.removeElementAt(0);

vec.removeAllElements();
```

```
// ================================
// 6. Size and Capacity
// ================================

int size = vec3.size();

int capacity = vec3.capacity();


// ================================
// 7. Capacity Operations
// ================================

vec3.ensureCapacity(20);

vec3.trimToSize();

vec3.setSize(10); // expand vector size
```

```java
// ================================
// 8. Searching Elements
// ================================

boolean contains = vec3.contains(3);

int index = vec3.indexOf(3);

int lastIndex = vec3.lastIndexOf(3);


// ================================
// 9. Checking Empty
// ================================

boolean empty = vec3.isEmpty();
```

```java
// ================================
// 10. Iterating Vector
// ================================

// For Loop
for(int i=0;i<vec3.size();i++)
{
    Integer val = vec3.get(i);
}

// ForEach Loop
for(Integer num : vec3)
{
    Integer val = num;
}
```

```
// Iterator
Iterator<Integer> iterator = vec3.iterator();

while(iterator.hasNext())
{
    Integer val = iterator.next();
}

// Enumeration (legacy)
Enumeration<Integer> enumeration = vec3.elements();

while(enumeration.hasMoreElements())
{
    Integer val = enumeration.nextElement();
}
```

```
// ================================
// 11. Sorting
// ================================

Collections.sort(vec3);

Collections.sort(vec3,Collections.reverseOrder());
```

```
    // ================================
    // 12. Reversing
    // ================================

    Collections.reverse(vec3);


    // ================================
    // 13. Shuffling
    // ================================

    Collections.shuffle(vec3);


    // ================================
    // 14. SubList
    // ================================

    List<Integer> subList = vec3.subList(1,3);
```

```
// ================================
// 15. Converting Vector
// ================================

Object[] array = vec3.toArray();

Integer[] intArray = vec3.toArray(new Integer[0]);


// ================================
// 16. Cloning
// ================================

Vector<Integer> cloned = (Vector<Integer>) vec3.clone();
```

```
// ================================
// 17. Clearing Vector
// ================================

vec3.clear();


// ================================
// 18. Bulk Operations
// ================================

vec4.addAll(Arrays.asList(2,4,6,8));

vec4.replaceAll(n -> n*2);

vec4.removeIf(n -> n>10);
```

```
// ================================
// 19. Retain Operations
// ================================

Vector<Integer> retain = new Vector<>(Arrays.asList(4,8));

vec4.retainAll(retain);

boolean containsAll = vec4.containsAll(retain);


// ================================
// 20. Copy Into Array
// ================================

Integer[] arr = new Integer[vec4.size()];

vec4.copyInto(arr);
```

ArrayList vs Vector

Feature	ArrayList	Vector
Synchronization	Not synchronized	Synchronized (thread-safe)
Performance	Faster because no synchronization overhead	Slower due to synchronization
Thread Safety	Not thread-safe by default	Thread-safe
Introduced In	Java 1.2 (Collections Framework)	Java 1.0 (Legacy class)
Usage Today	Widely used in modern applications	Rarely used in modern applications
Concurrency Handling	Requires external synchronization (e.g., `Collections.synchronizedList`)	Built-in synchronization
Capacity Growth	Increases by **50%** when full	Doubles capacity or uses capacityIncrement
Iterator	Uses **fail-fast Iterator**	Uses **fail-fast Iterator** but also supports **Enumeration**
Legacy Methods	No legacy methods	Has legacy methods like `addElement()`, `elementAt()`
Enumeration Support	Not supported	Supported (`elements()`)
Part of Legacy Collection	No	Yes
Preferred In	Single-threaded or externally synchronized environments	Multi-threaded environments (though rarely used today)

STACK

```
// ================================
// 1. Creating Stack
// ================================

Stack<Integer> stack = new Stack<>();

Stack<Integer> stack2 = new Stack<>();


// ================================
// 2. Push Elements (Insert)
// ================================

stack.push(10);
stack.push(20);
stack.push(30);
stack.push(40);
```

```
// ================================
// 3. Peek (View Top Element)
// ================================

int topElement = stack.peek();
```

```
// ================================
// 4. Pop (Remove Top Element)
// ================================

int removed = stack.pop();
```

```
// ================================
// 5. Search Element
// ================================

int position = stack.search(20);
// Returns position from top


// ================================
// 6. Size
// ================================

int size = stack.size();
```

```
// ================================
// 7. Check if Empty
// ================================

boolean empty = stack.empty();


// ================================
// 8. Access Element by Index
// ================================

int element = stack.get(0);
```

```
// ================================
// 9. Update Element
// ================================

stack.set(0,100);


// ================================
// 10. Remove Element by Index
// ================================

stack.remove(1);
```

```
// ================================
// 11. Remove Element by Value
// ================================

stack.remove(Integer.valueOf(30));


// ================================
// 12. Add Multiple Elements
// ================================

stack.addAll(Arrays.asList(50,60,70));
```

```
 // ================================
 // 13. Contains
 // ================================

 boolean contains = stack.contains(60);


 // ================================
 // 14. Index Operations
 // ================================

 int index = stack.indexOf(60);

 int lastIndex = stack.lastIndexOf(60);
```

```
// ================================
// 15. Iterating Stack
// ================================

// For Loop
for(int i=0;i<stack.size();i++)
{
    Integer val = stack.get(i);
}
```

```java
// ForEach Loop
for(Integer num : stack)
{
    Integer val = num;
}

// Iterator
Iterator<Integer> iterator = stack.iterator();

while(iterator.hasNext())
{
    Integer val = iterator.next();
}
```

```java
// ListIterator
ListIterator<Integer> listIterator = stack.listIterator();

while(listIterator.hasNext())
{
    Integer val = listIterator.next();
}


// ================================
// 16. Sorting Stack
// ================================

Collections.sort(stack);

Collections.sort(stack,Collections.reverseOrder());
```

```java
// ================================
// 17. Reversing
// ================================

Collections.reverse(stack);


// ================================
// 18. Shuffle
// ================================

Collections.shuffle(stack);
```

```java
// ================================
// 19. SubList
// ================================

List<Integer> subList = stack.subList(1,3);


// ================================
// 20. Convert Stack to Array
// ================================

Object[] array = stack.toArray();

Integer[] arr2 = stack.toArray(new Integer[0]);


// ================================
// 21. Clone Stack
// ================================

Stack<Integer> clonedStack = (Stack<Integer>) stack.clone();
```

```java
// ================================
// 23. Replace All
// ================================

stack.replaceAll(n -> n*2);


// ================================
// 24. Retain All
// ================================

stack2.addAll(Arrays.asList(20,40));

stack.retainAll(stack2);


// ================================
// 25. Clear Stack
// ================================

stack.clear();
```

HASHMAP

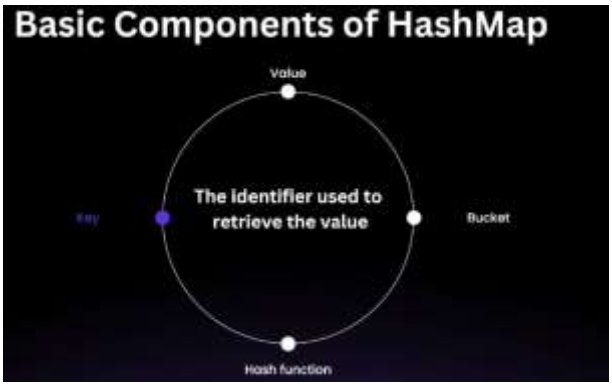
In Java, a Map is an object that maps keys to values. It cannot contain duplicate keys, and each key can map to at most one value. *Think of it as a dictionary* where you look up a word (key) to find its definition (value).

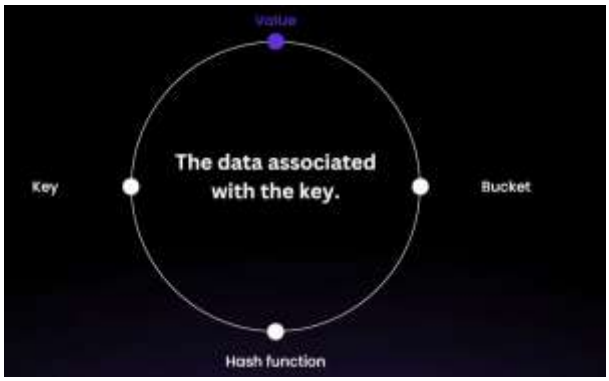
Roll Number	Name
101	Alice Johnson
102	Bob Smith
103	Charlie Brown
104	David Williams
105	Eva Thompson

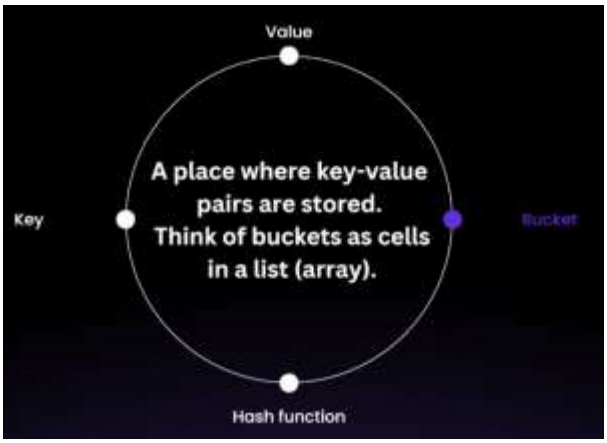
Key Characteristics

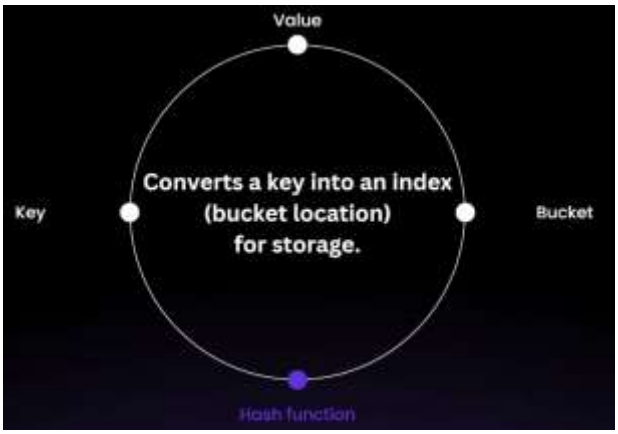
Unordered: Does not maintain any order of its elements.
Allows null Keys and Values: Can have one null key and multiple null values.
Not Synchronized: Not thread-safe; requires external synchronization if used in a multi-threaded context.
Performance: Offers constant-time performance (O(1)) for basic operations like get and put, assuming the hash function disperses elements properly.

Internal Structure of HashMap









A hash function is an algorithm that takes an input (or "key") and returns a fixed-size string of bytes, typically a numerical value. The output is known as a hash code, hash value, or simply hash. ***The primary purpose of a hash function is to map data of arbitrary size to data of fixed size***

- ***Deterministic:*** **The same input will always produce the same output.**
- ***Fixed Output Size:*** **Regardless of the input size, the hash code has a consistent size (e.g., 32-bit, 64-bit).**
- ***Efficient Computation:*** **The hash function should compute the hash quickly.**

How Data is Stored in HashMap

Step 1: Hashing the Key

First, the key is passed through a hash function to generate a unique hash code (an integer number). This hash code helps determine where the key-value pair will be stored in the array (called a "bucket array").

Step 2: Calculating the Index

The hash code is then used to calculate an index in the array (bucket location) using

int index = hashCode % arraySize;

The index decides which bucket will hold this key-value pair.

For example, if the array size is 16, the key's hash code will be divided by 16, and the remainder will be the index.

Step 3: Storing in the Bucket

The key-value pair is stored in the bucket at the calculated index. Each bucket can hold multiple key-value pairs

(this is called a collision handling mechanism, discussed later).

map.put("apple", 50);

- "apple" is the key.
- 50 is the value.
- The hash code of "apple" is calculated.
- The index is found using the hash code.
- The pair ("apple", 50) is stored in the corresponding bucket.

How HashMap Retrieves Data

When we call get(key), the HashMap follows these steps:

Hashing the Key: Similar to insertion, the key is hashed using the same hash function to calculate its hash code.

Finding the Index: The hash code is used to find the index of the bucket where the key-value pair is stored.

Searching in the Bucket: Once the correct bucket is found, it checks for the key in that bucket. If it finds the key, it returns the associated value.

Handling Collisions

Since different keys can generate the same index (called a collision), HashMap uses a technique to handle this situation. Java's HashMap uses Linked Lists (or balanced trees after Java 8) for this.

If multiple key-value pairs map to the same bucket, they are stored in a linked list inside the bucket.

When a key-value pair is retrieved, the HashMap traverses the linked list, checking each key until it finds a match.

HashMap Resizing (Rehashing)

HashMap has an internal array size, which by default is 16.
When the number of elements (key-value pairs) grows and exceeds a certain load factor (default is 0.75), HashMap automatically resizes the array to hold more data. This process is called rehashing.

The default size of the array is 16, so when more than 12 elements (16 * 0.75) are inserted, the HashMap will resize.

During rehashing

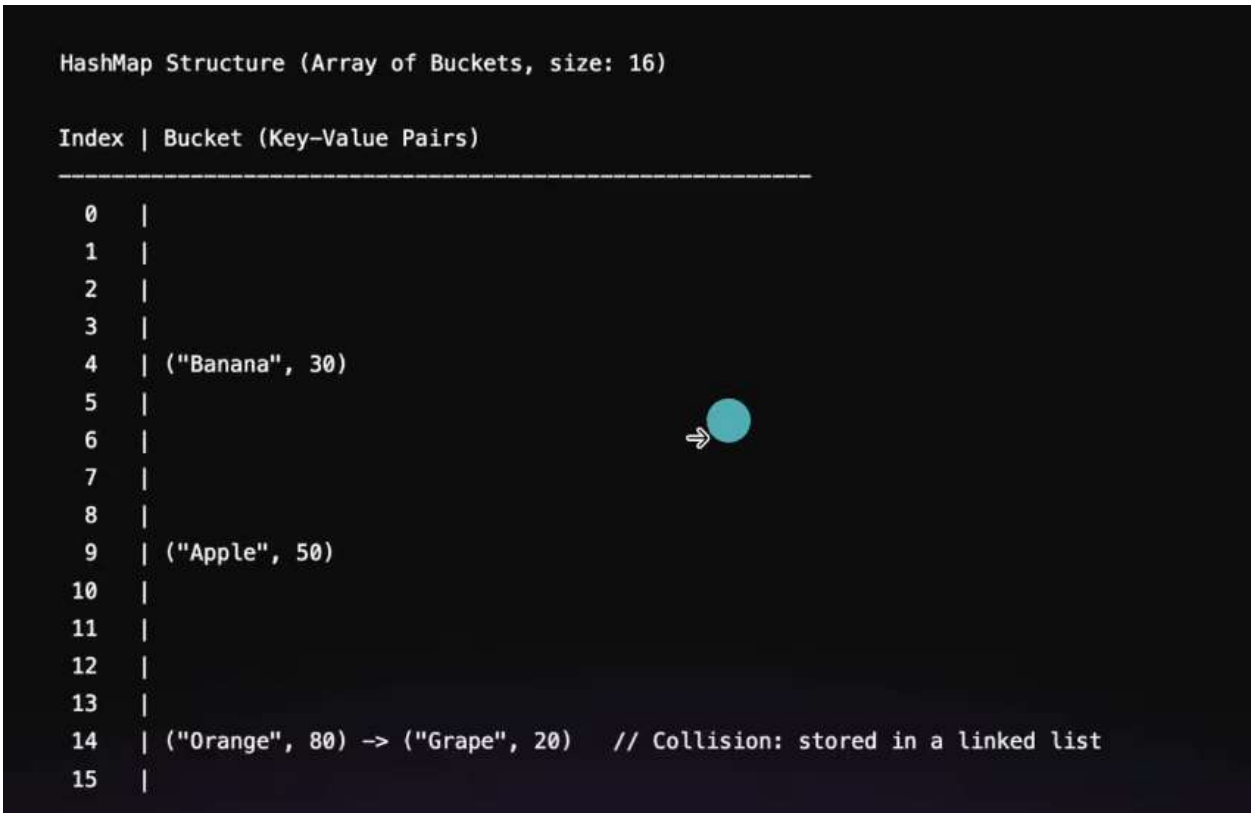
The array size is doubled.

1. All existing entries are rehashed (i.e., their positions are recalculated) and placed into the new array.
2. This ensures the HashMap continues to perform efficiently even as more data is added.

Time Complexity

HashMap provides constant time O(1) performance for basic operations like put() and get() **(assuming no collisions). However, if there are many collisions, and many entries are stored in the same bucket, the performance can degrade to O(n), where n is the number of elements in that bucket.**

But after Java 8, if there are too many elements in a bucket, HashMap switches to a balanced tree instead of a linked list to ensure better performance O(log n).



Operation	Average-Case Time Complexity	Worst-Case Time Complexity	Explanation
put(key, value)	O(1)	O(log n)	Inserts a key-value pair. Average: Constant time due to direct bucket access. Worst-Case: O(log n) when bucket converts to a Red-Black Tree after exceeding collision threshold.
get(key)	O(1)	O(log n)	Retrieves the value associated with a key. Average: Constant time via direct bucket access. Worst-Case: O(log n) when searching within a treeified bucket.
remove(key)	O(1)	O(log n)	Removes the key-value pair associated with a key. Average: Constant time with direct access. Worst-Case: O(log n) when removing from a treeified bucket.
containsKey(key)	O(1)	O(log n)	Checks if a key exists in the map. Average: Constant time via direct bucket access. Worst-Case: O(log n) when searching within a treeified bucket.
containsValue(value)	O(n)	O(n)	Checks if a value exists in the map. Both average and worst-case are linear time since it may need to traverse all entries.
size()	O(1)	O(1)	Returns the number of key-value pairs. Both average and worst-case are constant time as the size is maintained as a separate field.

```java
// ================================
// 1. Creating HashMap
// ================================

HashMap<Integer,String> map = new HashMap<>();

HashMap<Integer,String> map2 = new HashMap<>(10);

HashMap<Integer,String> map3 = new HashMap<>(16,0.75f);
```

```java
 // ================================
 // 2. Adding Elements
 // ================================

 map.put(1,"Surya");
 map.put(2,"Mahesh");
 map.put(3,"Kiran");

 map.put(4,"Anil");


 // ================================
 // 3. Accessing Elements
 // ================================

 String value = map.get(1);

 String defaultValue = map.getOrDefault(10,"Not Found");
```

```java
// ================================
// 4. Updating Value
// ================================

map.put(2,"Mahesh Updated");
```

```java
// ================================
// 5. Checking Keys / Values
// ================================

boolean hasKey = map.containsKey(3);

boolean hasValue = map.containsValue("Surya");


// ================================
// 6. Remove Elements
// ================================

map.remove(4);

map.remove(2,"Mahesh Updated");


// ================================
// 7. Size
// ================================

int size = map.size();
```

```java
// ================================
// 9. Iterating HashMap
// ================================

// Iterate Keys
for(Integer key : map.keySet())
{
    String val = map.get(key);
}
```

```java
// Iterate Values
for(String val : map.values())
{
    String value2 = val;
}


// Iterate Entries
for(Map.Entry<Integer,String> entry : map.entrySet())
{
    Integer key = entry.getKey();
    String val = entry.getValue();
}


// Iterator Example
Iterator<Map.Entry<Integer,String>> iterator =
        map.entrySet().iterator();

while(iterator.hasNext())
{
    Map.Entry<Integer,String> entry = iterator.next();
}
```

```java
// ================================
// 10. Replace Values
// ================================

map.replace(1,"Surya Updated");

map.replace(1,"Surya Updated","Surya Final");


// ================================
// 11. Put If Absent
// ================================

map.putIfAbsent(5,"New Employee");
```

```
// ==============================
// 8. Check if Empty
// ==============================

boolean empty = map.isEmpty();
```

```
// ==============================
// 12. Compute Methods
// ==============================

map.compute(1,(k,v)->v+" Kumar");

map.computeIfAbsent(6,k->"Generated Value");

map.computeIfPresent(3,(k,v)->v+" Present");


// ==============================
// 13. Merge Values
// ==============================

map.merge(1," Extra",(oldVal,newVal)->oldVal+newVal);
```

```
// ==============================
// 14. Copy HashMap
// ==============================

HashMap<Integer,String> copyMap = new HashMap<>(map);


// ==============================
// 15. Put All
// ==============================

map2.putAll(map);
```

```java
// ================================
// 16. Get KeySet / Values / EntrySet
// ================================

Set<Integer> keys = map.keySet();

Collection<String> values = map.values();

Set<Map.Entry<Integer,String>> entries = map.entrySet();


// ================================
// 17. Convert Map to List
// ================================

List<Integer> keyList = new ArrayList<>(map.keySet());

List<String> valueList = new ArrayList<>(map.values());
```

```java
// ================================
// 18. Sorting HashMap by Keys
// ================================

TreeMap<Integer,String> sortedByKeys =
        new TreeMap<>(map);


// ================================
// 19. Sorting by Values
// ================================

List<Map.Entry<Integer,String>> sortedByValue =
        new ArrayList<>(map.entrySet());

sortedByValue.sort(Map.Entry.comparingByValue());
```

```
    // ==============================
    // 20. Clearing Map
    // ==============================

    map.clear();
```

HashMap Internal Working – Collision Handling

Case 1 — Same Key in HashMap

```
Map<Integer,String> m = new HashMap<>();

m.put(1,"Mahesh");
m.put(1,"Surya");

System.out.println(m);
```

Output

```
{1=Surya}
```

What Happens Internally

1. First insert

```
1 → Mahesh
```

2. Second insert with same key

- `HashMap` checks **hashCode()**
- Finds **same bucket**
- Calls **equals() on key**
- Keys are equal (`1 == 1`)

So **value is replaced**

```
1 → Surya
```

Case 2 — Custom Object Key

```
Map<Employee,String> m = new HashMap<>();

m.put(new Employee("Mahesh",1),"Mahesh");
m.put(new Employee("Mahesh",1),"Surya");

System.out.println(m);
```

Output (if equals/hashCode NOT overridden)

```
{Employee@6d06d69c=Mahesh, Employee@7852e922=Surya}
```

What Happens Internally

1. First object inserted

```
Employee("Mahesh",1) → Mahesh
```

2. Second object inserted

- New object → different **hashCode**
- Different **memory reference**

So HashMap treats it as **different key**

- ✔ Both entries exist
- ✔ No replacement happens

Case 3 — If equals() and hashCode() Implemented

```
class Employee {

    String name;
    int id;

    Employee(String name,int id){
        this.name=name;
        this.id=id;
    }

    public boolean equals(Object o){
        Employee e=(Employee)o;
        return this.id==e.id && this.name.equals(e.name);
    }

    public int hashCode(){
        return Objects.hash(name,id);
    }
}
```

```
m.put(new Employee("Mahesh",1),"Mahesh");
m.put(new Employee("Mahesh",1),"Surya");
```

Output

```
{Employee(Mahesh,1)=Surya}
```

Internal Flow

1. HashMap computes **hashCode()**
2. Same hash → same bucket
3. Calls **equals()**
4. Keys match → value replaced

HashMap replaces a value only when both **hashCode() and equals() of the key match**. Otherwise it treats the object as a new key and stores another entry.

LinkedHashMap

```
    // 1️⃣ Create LinkedHashMap
    LinkedHashMap<Integer, String> map = new LinkedHashMap<>();

    // 2️⃣ Insert elements (put)
    map.put(1, "Java");
    map.put(2, "Spring");
    map.put(3, "Angular");
    map.put(4, "MySQL");

    System.out.println("Initial Map: " + map);

    // 3️⃣ Access value using key
    System.out.println("Value for key 2: " + map.get(2));

    // 4️⃣ Check if key exists
    System.out.println("Contains key 3: " + map.containsKey(3));

    // 5️⃣ Check if value exists
    System.out.println("Contains value 'Java': " + map.containsValue("Java"));
```

```
// 6️⃣ Update value
map.put(2, "Spring Boot");
System.out.println("After updating key 2: " + map);

// 7️⃣ Remove element
map.remove(4);
System.out.println("After removing key 4: " + map);

// 8️⃣ Size of map
System.out.println("Size: " + map.size());

// 9️⃣ Iterate using keySet
System.out.println("\nIterating using keySet:");
for(Integer key : map.keySet()) {
    System.out.println(key + " -> " + map.get(key));
}
```

```java
// 🔟 Iterate using entrySet (best practice)
System.out.println("\nIterating using entrySet:");
for(Map.Entry<Integer, String> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}

// 1️⃣1️⃣ Iterate using forEach
System.out.println("\nIterating using forEach:");
map.forEach((k,v) -> System.out.println(k + " -> " + v));

// 1️⃣2️⃣ Get all keys
System.out.println("Keys: " + map.keySet());

// 1️⃣3️⃣ Get all values
System.out.println("Values: " + map.values());

// 1️⃣4️⃣ Check if map is empty
System.out.println("Is map empty? " + map.isEmpty());

// 1️⃣5️⃣ Clear map
map.clear();
System.out.println("After clear(): " + map);
```

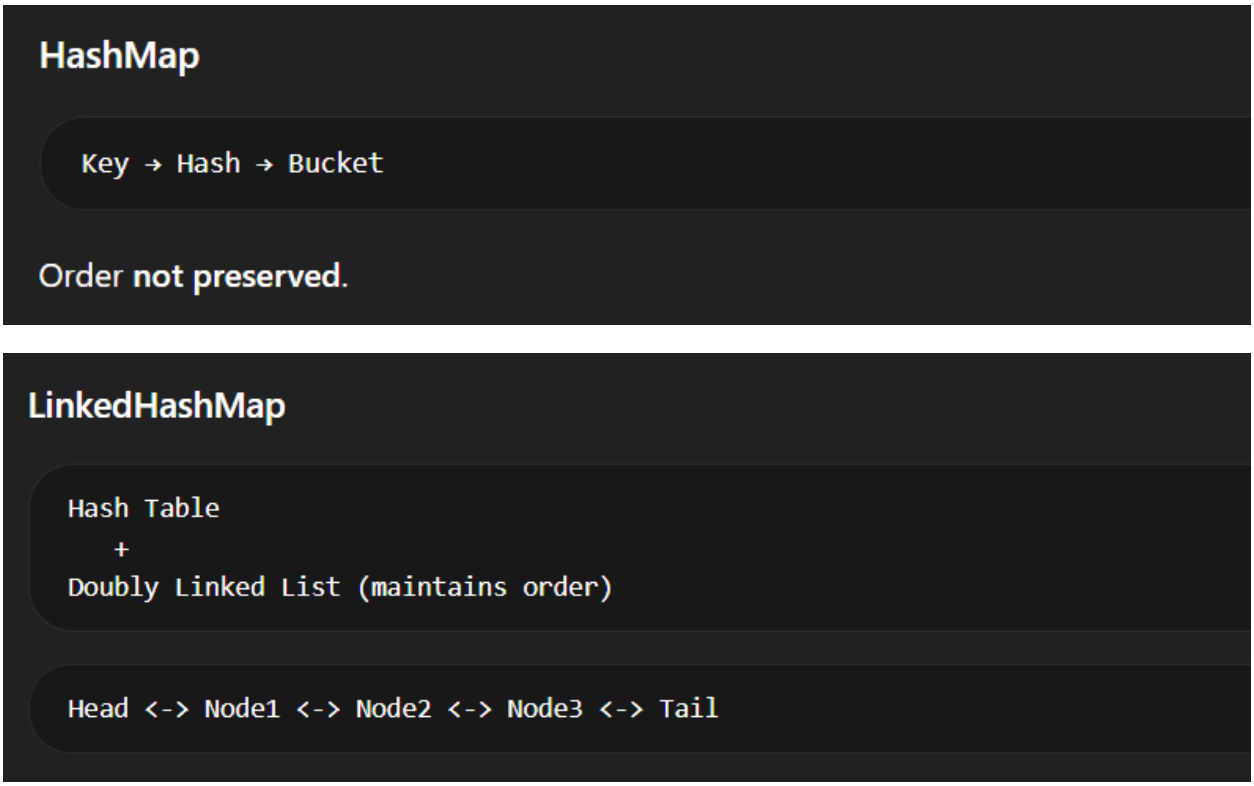
Key Property of LinkedHashMap

LinkedHashMap maintains insertion order.

HashMap vs LinkedHashMap

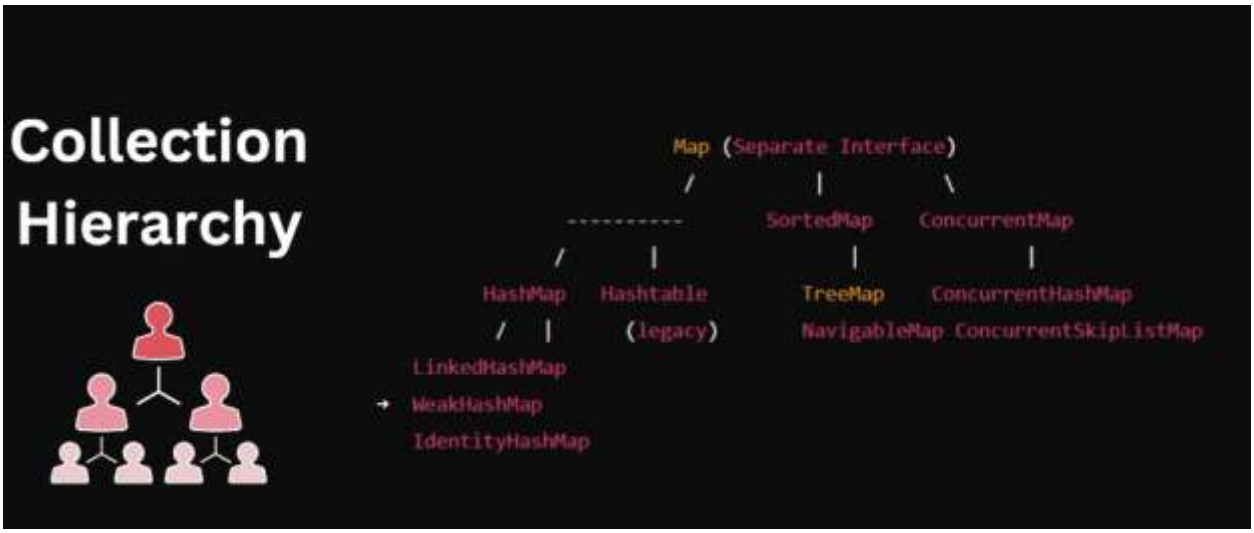
Feature	HashMap	LinkedHashMap
Order	No order guaranteed	Maintains insertion order
Internal Structure	Hash table	Hash table + Doubly linked list
Iteration Order	Random	Same as insertion
Performance	Slightly faster	Slightly slower (due to linked list)
Memory Usage	Less	More
Access Order Option	Not supported	Supported (LRU cache usage)

Internal Structure Visualization



HashMap vs LinkedHashMap

HashMap does not guarantee any order of elements, whereas LinkedHashMap maintains insertion order using a doubly linked list along with the hash table.



HashMap vs LinkedHashMap vs TreeMap

```
Map<Integer, String> hashMap = new HashMap<>();
Map<Integer, String> linkedHashMap = new LinkedHashMap<>();
Map<Integer, String> treeMap = new TreeMap<>();

int[] keys = {30, 10, 50, 20, 40};

// Insert elements
for(int key : keys) {
    hashMap.put(key, "Val" + key);
    linkedHashMap.put(key, "Val" + key);
    treeMap.put(key, "Val" + key);
}

System.out.println("HashMap: " + hashMap);
System.out.println("LinkedHashMap: " + linkedHashMap);
System.out.println("TreeMap: " + treeMap);

// get()
System.out.println("\nGet key 20:");
System.out.println("HashMap: " + hashMap.get(20));
System.out.println("LinkedHashMap: " + linkedHashMap.get(20));
System.out.println("TreeMap: " + treeMap.get(20));
```

```
// containsKey()
System.out.println("\nContains key 30:");
System.out.println("HashMap: " + hashMap.containsKey(30));
System.out.println("LinkedHashMap: " + linkedHashMap.containsKey(30));
System.out.println("TreeMap: " + treeMap.containsKey(30));

// remove()
hashMap.remove(50);
linkedHashMap.remove(50);
treeMap.remove(50);
```

Example Output (Key Difference)

```
HashMap: {50=Val50, 20=Val20, 40=Val40, 10=Val10, 30=Val30}
LinkedHashMap: {30=Val30, 10=Val10, 50=Val50, 20=Val20, 40=Val40}
TreeMap: {10=Val10, 20=Val20, 30=Val30, 40=Val40, 50=Val50}
```

- **HashMap → Random order**
- **LinkedHashMap → Insertion order**
- **TreeMap → Sorted order**

HashMap vs LinkedHashMap vs TreeMap

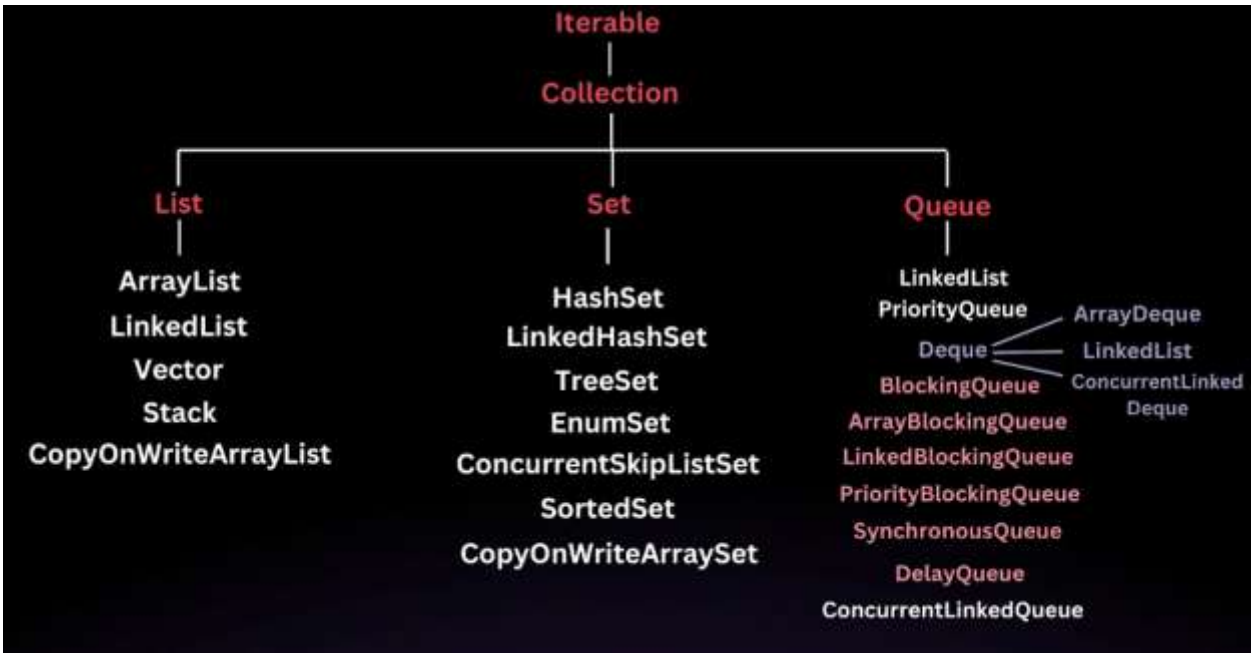
Feature	HashMap	LinkedHashMap	TreeMap
Order	No order guaranteed	Maintains insertion order	Sorted by key
Underlying DS	Hash Table	Hash Table + Doubly Linked List	Red-Black Tree
Null Key	One null key allowed	One null key allowed	❌ No null key
Null Values	Allowed	Allowed	Allowed
Iteration Order	Random	Insertion order	Sorted order
Performance	Fastest	Slightly slower	Slowest
Extra Features	Basic map	Access order option (LRU cache)	Navigable methods

Time Complexities

Operation	HashMap	LinkedHashMap	TreeMap
put()	O(1)	O(1)	O(log n)
get()	O(1)	O(1)	O(log n)
remove()	O(1)	O(1)	O(log n)
containsKey()	O(1)	O(1)	O(log n)
iteration	O(n)	O(n)	O(n)

HashMap stores key-value pairs without any ordering, LinkedHashMap maintains insertion order using a linked list, and TreeMap keeps keys sorted using a Red-Black Tree with O(log n) operations.

SETS



HashSet

```
// 1 Create HashSet
HashSet<String> set = new HashSet<>();

// 2 Add elements
set.add("Java");
set.add("Spring");
set.add("Angular");
set.add("MySQL");

System.out.println("Initial Set: " + set);

// 3 Add duplicate element
set.add("Java"); // duplicates ignored
System.out.println("After adding duplicate: " + set);

// 4 Check if element exists
System.out.println("Contains 'Java'? " + set.contains("Java"));
```

```java
// 5️⃣ Remove element
set.remove("Angular");
System.out.println("After removing Angular: " + set);

// 6️⃣ Size of set
System.out.println("Size: " + set.size());

// 7️⃣ Iterate using for-each
System.out.println("\nIterating using for-each:");
for(String s : set) {
    System.out.println(s);
}

// 8️⃣ Iterate using iterator
System.out.println("\nIterating using Iterator:");
Iterator<String> it = set.iterator();
while(it.hasNext()) {
    System.out.println(it.next());
}
```

```java
// 9️⃣ Convert Set to Array
Object[] arr = set.toArray();
System.out.println("\nArray form: " + Arrays.toString(arr));

// 🔟 Check if empty
System.out.println("Is set empty? " + set.isEmpty());

// 1️⃣1️⃣ Clear the set
set.clear();
System.out.println("After clear(): " + set);
```

HashSet is a collection that stores unique elements and does not maintain insertion order. It internally uses a HashMap for storing elements.

HashSet vs LinkedHashSet vs TreeSet

```
// Creating sets
Set<Integer> hashSet = new HashSet<>();
Set<Integer> linkedHashSet = new LinkedHashSet<>();
Set<Integer> treeSet = new TreeSet<>();

// Elements to insert
int[] numbers = {30, 10, 50, 20, 40, 10};

// Adding elements
for(int num : numbers) {
    hashSet.add(num);
    linkedHashSet.add(num);
    treeSet.add(num);
}

System.out.println("HashSet: " + hashSet);
System.out.println("LinkedHashSet: " + linkedHashSet);
System.out.println("TreeSet: " + treeSet);
```

```
// contains operation
System.out.println("\nContains 20:");
System.out.println("HashSet: " + hashSet.contains(20));
System.out.println("LinkedHashSet: " + linkedHashSet.contains(20));
System.out.println("TreeSet: " + treeSet.contains(20));

// remove operation
hashSet.remove(30);
linkedHashSet.remove(30);
treeSet.remove(30);

System.out.println("\nAfter removing 30:");
System.out.println("HashSet: " + hashSet);
System.out.println("LinkedHashSet: " + linkedHashSet);
System.out.println("TreeSet: " + treeSet);
```

```java
// iteration
System.out.println("\nIteration:");
System.out.println("HashSet: ");
for(Integer n : hashSet) System.out.print(n + " ");

System.out.println("\nLinkedHashSet: ");
for(Integer n : linkedHashSet) System.out.print(n + " ");

System.out.println("\nTreeSet: ");
for(Integer n : treeSet) System.out.print(n + " ");
```

Explanation:

- **HashSet** → Random order
- **LinkedHashSet** → Insertion order
- **TreeSet** → Sorted order

HashSet vs LinkedHashSet vs TreeSet

Feature	HashSet	LinkedHashSet	TreeSet
Order	No order	Maintains insertion order	Sorted order
Underlying DS	Hash Table	Hash Table + Doubly Linked List	Red-Black Tree
Duplicates	Not allowed	Not allowed	Not allowed
Null values	Allows one null	Allows one null	❌ Does not allow null
Performance	Fastest	Slightly slower	Slowest
Navigation methods	No	No	Yes (first, last, higher, lower)
Sorting	Not supported	Not supported	Automatically sorted

Time Complexities

Operation	HashSet	LinkedHashSet	TreeSet
add()	O(1)	O(1)	O(log n)
remove()	O(1)	O(1)	O(log n)
contains()	O(1)	O(1)	O(log n)
iteration	O(n)	O(n)	O(n)
search	O(1)	O(1)	O(log n)

HashSet stores unique elements with no ordering.

LinkedHashSet maintains insertion order using a linked list.

TreeSet stores elements in sorted order using a Red-Black Tree.

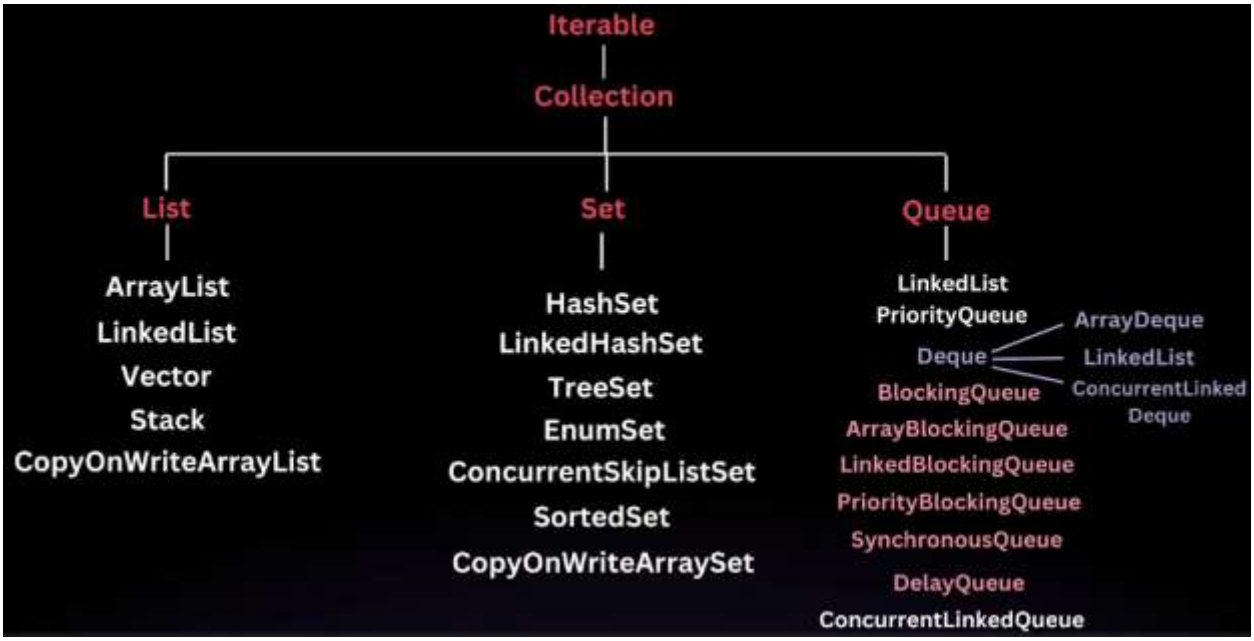
Java Collections – Time Complexity Cheat Sheet (Most Used)

Collection	add()	get()	remove()	contains()	iteration	Notes
ArrayList	O(1) amortized	O(1)	O(n)	O(n)	O(n)	Dynamic array
LinkedList	O(1)	O(n)	O(1)	O(n)	O(n)	Doubly linked list
HashSet	O(1)	—	O(1)	O(1)	O(n)	Uses HashMap internally
LinkedHashSet	O(1)	—	O(1)	O(1)	O(n)	Maintains insertion order
TreeSet	O(log n)	—	O(log n)	O(log n)	O(n)	Red-Black Tree (sorted)
HashMap	O(1)	O(1)	O(1)	O(1)	O(n)	Hash table
LinkedHashMap	O(1)	O(1)	O(1)	O(1)	O(n)	Maintains insertion order
TreeMap	O(log n)	O(log n)	O(log n)	O(log n)	O(n)	Red-Black Tree (sorted)
PriorityQueue	O(log n)	O(1) (peek)	O(log n)	O(n)	O(n)	Binary Heap

Internal Data Structures

Collection	Internal DS
ArrayList	Dynamic Array
LinkedList	Doubly Linked List
HashSet	HashMap
LinkedHashSet	HashMap + Linked List
TreeSet	Red-Black Tree
HashMap	Hash Table
LinkedHashMap	Hash Table + Linked List
TreeMap	Red-Black Tree
PriorityQueue	Binary Heap

Queue



```java
// Create Queue
Queue<Integer> queue = new LinkedList<>();

// 1️⃣ add() - insert element
queue.add(10);
queue.add(20);
queue.add(30);

System.out.println("Queue after add(): " + queue);

// 2️⃣ offer() - safe insertion
queue.offer(40);

System.out.println("Queue after offer(): " + queue);

// 3️⃣ peek() - view front element
System.out.println("Front element (peek): " + queue.peek());

// 4️⃣ element() - similar to peek
System.out.println("Front element (element): " + queue.element());
```

```java
// 5️⃣ poll() - remove front safely
System.out.println("Removed using poll(): " + queue.poll());

System.out.println("Queue after poll(): " + queue);

// 6️⃣ remove() - remove head
System.out.println("Removed using remove(): " + queue.remove());

System.out.println("Queue after remove(): " + queue);

// 7️⃣ contains
System.out.println("Contains 30? " + queue.contains(30));

// 8️⃣ size
System.out.println("Queue size: " + queue.size());
```

```java
// 9️⃣ iteration
System.out.println("Iterating queue:");

for(Integer n : queue) {
    System.out.println(n);
}

// 🔟 check empty
System.out.println("Is empty? " + queue.isEmpty());
```

```
Queue after add(): [10, 20, 30]
Queue after offer(): [10, 20, 30, 40]
Front element (peek): 10
Removed using poll(): 10
Queue after poll(): [20, 30, 40]
Removed using remove(): 20
Queue after remove(): [30, 40]
```

Time Complexities (Queue using LinkedList)

Operation	Time
add / offer	O(1)
remove / poll	O(1)
peek / element	O(1)
contains	O(n)
iteration	O(n)

Queue Vs ArrayBlockingQueue

```
Queue<Integer> q = new ArrayBlockingQueue<>(3);

q.add(10);
q.add(20);
q.add(30);

System.out.println("Queue: " + q);

// remove head
System.out.println("Removed: " + q.remove());

// view head
System.out.println("Peek: " + q.peek());
```

```
// ================================
// 2. ArrayBlockingQueue Creation
// ================================

ArrayBlockingQueue<Integer> abq = new ArrayBlockingQueue<>(3);

abq.add(1);
abq.add(2);
abq.add(3);

System.out.println("ArrayBlockingQueue: " + abq);
```

```
// ================================
// 3. Offer vs Add
// ================================

boolean result = abq.offer(4);   // returns false if full
System.out.println("Offer result when full: " + result);

try {
    abq.add(4);  // throws exception if full
} catch(Exception e){
    System.out.println("Add throws exception when full");
}
```

```java
// ================================
// 4. Peek vs Element
// ================================

System.out.println("Peek: " + abq.peek());

System.out.println("Element: " + abq.element());
```

```java
// ================================
// 5. Poll vs Remove
// ================================

System.out.println("Poll: " + abq.poll());

System.out.println("Remove: " + abq.remove());
```

```java
// ================================
// 6. Blocking Operations
// ================================

ArrayBlockingQueue<Integer> blockingQueue =
        new ArrayBlockingQueue<>(2);

blockingQueue.put(100);
blockingQueue.put(200);

System.out.println("BlockingQueue: " + blockingQueue);

// take waits if queue empty
int val = blockingQueue.take();

System.out.println("Take element: " + val);
```

```java
// ================================
// 7. Remaining Capacity
// ================================

System.out.println(
        "Remaining Capacity: " +
        blockingQueue.remainingCapacity()
);
```

```java
// ================================
// 8. Iteration
// ================================

for(Integer num : blockingQueue)
{
    System.out.println("Value: " + num);
}
```

```java
// ================================
// 9. Clear Queue
// ================================

blockingQueue.clear();

System.out.println("Queue after clear: " + blockingQueue);
```

PriorityQueue

```java
// ================================
// 1. Creating PriorityQueue
// ================================

PriorityQueue<Integer> pq = new PriorityQueue<>();

PriorityQueue<Integer> pq2 = new PriorityQueue<>(10);

PriorityQueue<Integer> pq3 = new PriorityQueue<>(Arrays.asList(5,1,8,3));
```

```java
// ================================
// 2. Adding Elements
// ================================

pq.add(10);
pq.add(5);
pq.add(20);

pq.offer(15);
```

```java
// ================================
// 3. View Head Element
// ================================

int head = pq.peek();

int head2 = pq.element();


// ================================
// 4. Remove Head Element
// ================================

int removed1 = pq.poll();

int removed2 = pq.remove();
```

```java
// ================================
// 5. Size
// ================================

int size = pq.size();


// ================================
// 6. Check if Empty
// ================================

boolean empty = pq.isEmpty();


// ================================
// 7. Search Element
// ================================

boolean contains = pq.contains(10);
```

```java
// ================================
// 8. Iterating PriorityQueue
// ================================

for(Integer num : pq)
{
    Integer val = num;
}


Iterator<Integer> iterator = pq.iterator();

while(iterator.hasNext())
{
    Integer val = iterator.next();
}
```

```java
// ================================
// 9. Min Heap (Default Behavior)
// ================================

PriorityQueue<Integer> minHeap = new PriorityQueue<>();

minHeap.add(30);
minHeap.add(10);
minHeap.add(20);

int min = minHeap.peek();
```

```java
// ================================
// 10. Max Heap using Comparator
// ================================

PriorityQueue<Integer> maxHeap =
        new PriorityQueue<>(Comparator.reverseOrder());

maxHeap.add(30);
maxHeap.add(10);
maxHeap.add(20);

int max = maxHeap.peek();
```

```
// ================================
// 11. Custom Comparator
// ================================

PriorityQueue<Integer> customPQ =
        new PriorityQueue<>((a,b)->b-a);

customPQ.add(5);
customPQ.add(1);
customPQ.add(7);


// ================================
// 12. Add Multiple Elements
// ================================

pq.addAll(Arrays.asList(40,50,60));
```

```
// ================================
// 13. Remove Specific Element
// ================================

pq.remove(40);


// ================================
// 14. Convert to Array
// ================================

Object[] arr = pq.toArray();

Integer[] arr2 = pq.toArray(new Integer[0]);


// ================================
// 15. Clear PriorityQueue
// ================================

pq.clear();
```

```java
// ==============================
// 16. Bulk Operations
// ==============================

PriorityQueue<Integer> pqA =
        new PriorityQueue<>(Arrays.asList(1,2,3,4));

PriorityQueue<Integer> pqB =
        new PriorityQueue<>(Arrays.asList(3,4));

pqA.removeAll(pqB);

pqA.retainAll(pqB);
```

```java
// ==============================
// 17. Poll Elements (Heap Order)
// ==============================

PriorityQueue<Integer> heap =
        new PriorityQueue<>(Arrays.asList(7,2,9,1,5));

while(!heap.isEmpty())
{
    int val = heap.poll();
}
```

Operations Covered

Category	Methods
Creation	`PriorityQueue()`
Insert	`add()`, `offer()`
Remove	`poll()`, `remove()`
View head	`peek()`, `element()`
Search	`contains()`
Size	`size()`

Check empty	`isEmpty()`
Iteration	`iterator()`
Bulk operations	`addAll()`, `removeAll()`, `retainAll()`
Conversion	`toArray()`
Clear	`clear()`
Heap behavior	`poll()` order

Important Behavior : PriorityQueue is a Min Heap by default

Time Complexity

Operation	Complexity
Insert	O(log n)
Remove	O(log n)
Peek	O(1)
Search	O(n)

Key Characteristics

Feature	PriorityQueue
Structure	Binary Heap
Order	Not sorted iteration
Head	Smallest element (default)
Duplicates	Allowed
Nulls	Not allowed
Thread Safe	No

Deque

```
INSERTION METHODS

addFirst(E e): Inserts the specified element at the front.
addLast(E e): Inserts the specified element at the end.
offerFirst(E e): Inserts the specified element at the front if possible.
offerLast(E e): Inserts the specified element at the end if possible.
```

```
REMOVAL METHODS

removeFirst(): Retrieves and removes the first element.
removeLast(): Retrieves and removes the last element.
pollFirst(): Retrieves and removes the first element, or returns null if empty.
pollLast(): Retrieves and removes the last element, or returns null if empty.
```

```
EXAMINATION METHODS

getFirst(): Retrieves, but does not remove, the first element.
getLast(): Retrieves, but does not remove, the last element.
peekFirst(): Retrieves, but does not remove, the first element, or returns null if empty.
peekLast(): Retrieves, but does not remove, the last element, or returns null if empty.
```

```
STACK METHODS

push(E e): Adds an element at the front (equivalent to addFirst(E e)).
pop(): Removes and returns the first element (equivalent to removeFirst())..
```

```java
// ================================
// 1. Creating Deque
// ================================

Deque<Integer> dq = new ArrayDeque<>();
```

```
// ================================
// 2. Adding Elements
// ================================

dq.add(10);
dq.addFirst(5);
dq.addLast(20);

dq.offer(25);
dq.offerFirst(1);
dq.offerLast(30);
```

```
// ================================
// 3. Viewing Elements
// ================================

int first = dq.getFirst();
int last = dq.getLast();

int peekFirst = dq.peekFirst();
int peekLast = dq.peekLast();

int peek = dq.peek();
```

```
// ================================
// 4. Removing Elements
// ================================

int removedFirst = dq.removeFirst();
int removedLast = dq.removeLast();

int pollFirst = dq.pollFirst();
int pollLast = dq.pollLast();

int poll = dq.poll();
```

```
// ================================
// 5. Stack Operations (LIFO)
// ================================

dq.push(100);
dq.push(200);

int stackTop = dq.pop();

int stackPeek = dq.peek();
```

```
    // ================================
    // 6. Queue Operations (FIFO)
    // ================================

    dq.offer(300);
    dq.offer(400);

    int queueHead = dq.poll();


    // ================================
    // 7. Size
    // ================================

    int size = dq.size();
```

```
  // ================================
  // 8. Check if Empty
  // ================================

  boolean empty = dq.isEmpty();


  // ================================
  // 9. Search Element
  // ================================

  boolean contains = dq.contains(100);
```

```
// ================================
// 10. Iterating Deque
// ================================

for(Integer val : dq)
{
    Integer num = val;
}

Iterator<Integer> iterator = dq.iterator();

while(iterator.hasNext())
{
    Integer num = iterator.next();
}
```

```
  Iterator<Integer> desc = dq.descendingIterator();

  while(desc.hasNext())
  {
      Integer num = desc.next();
  }
```

```
  // ================================
  // 11. Add Multiple Elements
  // ================================

  dq.addAll(Arrays.asList(10,20,30));


  // ================================
  // 12. Remove Specific Element
  // ================================

  dq.remove(20);
```

```java
// ================================
// 13. Convert to Array
// ================================

Object[] arr = dq.toArray();

Integer[] arr2 = dq.toArray(new Integer[0]);


// ================================
// 14. Clear Deque
// ================================

dq.clear();
```

Operations Covered

Category	Methods
Creation	`new ArrayDeque()`
Insert	`add()`, `addFirst()`, `addLast()`
Offer	`offer()`, `offerFirst()`, `offerLast()`
Remove	`remove()`, `removeFirst()`, `removeLast()`
Poll	`poll()`, `pollFirst()`, `pollLast()`
Peek	`peek()`, `peekFirst()`, `peekLast()`
Stack	`push()`, `pop()`
Queue	`offer()`, `poll()`
Search	`contains()`

Iteration	`iterator()`, `descendingIterator()`
Bulk	`addAll()`
Conversion	`toArray()`
Clear	`clear()`

Key Characteristics of Deque

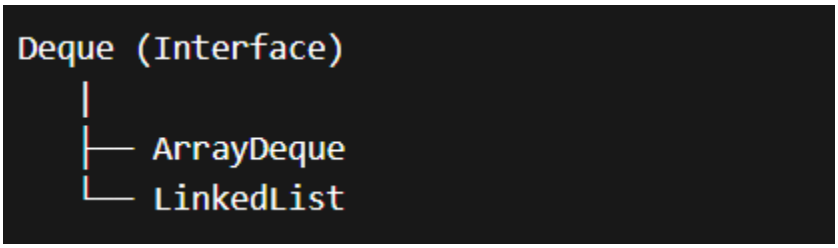
Feature	Deque
Meaning	Double Ended Queue
Insertion	Both front and rear
Deletion	Both front and rear
Null values	Not allowed
Thread-safe	No
Common implementation	`ArrayDeque`, `LinkedList`

Deque vs Queue

Feature	Queue	Deque
Insert	Rear only	Front & Rear
Remove	Front only	Front & Rear
Structure	FIFO	FIFO + LIFO

Deque Use Cases

Use Case	Example
Stack implementation	`push()` / `pop()`
Queue implementation	`offer()` / `poll()`
Sliding window problems	DSA
Palindrome checks	Algorithms
Undo/Redo operations	Applications




```java
Deque<Integer> dq = new LinkedList<>();

dq.addFirst(10);
dq.addLast(20);

dq.offerFirst(5);
dq.offerLast(25);

System.out.println(dq);

dq.removeFirst();
dq.removeLast();

System.out.println(dq);

dq.push(100);
dq.pop();
```

LinkedList can be used as a Deque because it implements the Deque interface. However, ArrayDeque is generally preferred for better performance and lower memory overhead.

```java
Deque<Integer> deque1 = new ArrayDeque<>(); // faster iteration, low memory, no null allowed
deque1.addFirst( e: 10);
deque1.addLast( e: 20);
deque1.offerFirst( e: 5);
deque1.offerLast( e: 25);
// 5, 10, 20, 25
System.out.println(deque1);
System.out.println("First Element: " + deque1.getFirst()); // Outputs 5
System.out.println("Last Element: " + deque1.getLast());   // Outputs 25
deque1.removeFirst(); // Removes 5
deque1.pollLast();    // Removes 25
// Current Deque: [10, 20]
```

Iterable vs Iterator in Java

In Java Collections, Iterable and Iterator are used to traverse elements in a collection.

But they have different responsibilities.

Iterable

Iterable is an interface that represents a collection capable of returning an iterator.

It provides the method:

```java
Iterator<T> iterator()
```

This allows objects to be used in a for-each loop.

Example:

```java
Iterable<Integer> list = new ArrayList<>();
```

Iterator

Iterator is used to traverse elements one by one in a collection.

It provides methods like:

hasNext()
next()
remove()

```java
Iterator<Integer> it = list.iterator();
```

Java Program Demonstrating Iterable and Iterator

```
// Collection implementing Iterable
List<Integer> list = new ArrayList<>();

list.add(10);
list.add(20);
list.add(30);
list.add(40);


// ========================
// Using Iterable (for-each)
// ========================

System.out.println("Using Iterable (for-each loop)");

for(Integer num : list)
{
    System.out.println(num);
}
```

```
// ========================
// Using Iterator
// ========================

System.out.println("Using Iterator");

Iterator<Integer> it = list.iterator();

while(it.hasNext())
{
    Integer val = it.next();
    System.out.println(val);
}
```

```java
// ========================
// Removing elements using Iterator
// ========================

Iterator<Integer> removeIterator = list.iterator();

while(removeIterator.hasNext())
{
    int val = removeIterator.next();

    if(val == 20)
    {
        removeIterator.remove();
    }
}
```

Iterable vs Iterator (Differences)

Feature	Iterable	Iterator
Definition	Represents a collection that can be iterated	Used to iterate elements
Purpose	Provides iterator	Traverses elements
Method	`iterator()`	`hasNext()`, `next()`, `remove()`
Loop Support	Enables `for-each` loop	Used in manual iteration
Package	`java.lang`	`java.util`
Control	No control over traversal	Full control over traversal
Modification	Cannot remove elements	Can remove elements during iteration

```
Collection
   |
   └── implements Iterable
            |
            └── iterator() → returns Iterator
```

```
List → Iterable → iterator() → Iterator → next elements
```

Every Java Collection implements Iterable, which is why we can write:

```
for(Integer num : list)
{
    System.out.println(num);
}
```

Internally Java converts this to:

```
Iterator<Integer> it = list.iterator();
while(it.hasNext())
{
    Integer val = it.next();
}
```

Interview Question : How HashMap Works Internally in Java

HashMap is a hash table based data structure used to store key-value pairs.

It provides O(1) average time complexity for put() and get() operations.

Basic Structure of HashMap

Internally, HashMap uses an array of nodes (buckets).

```
HashMap
   |
   |---- Bucket[0]
   |---- Bucket[1]
   |---- Bucket[2]
   |---- Bucket[3]
   |---- Bucket[4]
   |---- Bucket[n]
```

Each bucket stores entries in the form:

Node<Key, Value>

Each node contains:

```
hash
key
value
next (linked list pointer)
```

HashMap Node Structure

Internally Java defines something like:

```
static class Node<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next;
}
```

Step-by-Step: put(key, value)

Example:

```
HashMap<Integer,String> map = new HashMap<>();

map.put(10,"Surya");
```

Step 1: Calculate HashCode

```
key.hashCode()
```

Step 2: Hash Function

Step 3: Find Bucket Index

Step 4: Insert Node

Case 1: Bucket Empty

Bucket[5] → null → Insert directly → Node(10, "Surya")

Time Complexity: O(1)

Case 2: Collision Occurs

Collision means two keys map to the same bucket.

Example:

```
Bucket[5]
   |
   | → (10,Surya)
   | → (26,Mahesh)
```

Here entries are stored in a LinkedList.

Case 3: LinkedList → Tree (Java 8 Optimization)

If collisions become large:

LinkedList length > 8

Java converts it to a Red-Black Tree. → LinkedList → Balanced Tree

Internal Flow Summary

```
put(key,value)

1. Compute hashCode
2. Apply hash function
3. Find bucket index
4. Insert node
5. Handle collision
6. Resize if needed
```

Interview One-Line Answer

HashMap stores key-value pairs using a hash table. The key's hashCode determines the bucket index where the entry is stored. Collisions are handled using a LinkedList, and if the list grows beyond a threshold, it is converted into a Red-Black Tree to maintain efficient lookup.